Owned entity types are entity types that are part of the owner and cannot exist without the owner. You can use common one-to-one or one-to-many relationships to model the owned entities, but EF Core provides a more convenient way called owned entity types. You can use the OwnsOne() or OwnsMany() method to define owned entity types, instead of using the HasOne() or HasMany() method.

****

```
    public static class InvoiceModelCreatingExtensions
    {
        public static void ConfigureInvoice(this ModelBuilder builder)
        {
            // using Fluent API
            builder.Entity<Invoice>(b =>
            {
                b.ToTable("Invoices");
                b.HasKey(i => i.Id);
                b.Property(p => p.Id).HasColumnName("Id");
                b.Property(p =>
p.InvoiceNumber).HasColumnName("InvoiceNumber").HasColumnType("varchar(32)").IsRequi
red();
                b.Property(p =>
p.ContactName).HasColumnName("ContactName").HasMaxLength(32).IsRequired();
                b.Property(p =>
p.Description).HasColumnName("Description").HasMaxLength(256);
                b.Property(p =>
p.Amount).HasColumnName("Amount").HasColumnType("decimal(18,2)").IsRequired();
                b.Property(p => p.Amount).HasColumnName("Amount").HasPrecision(18,
2);
                b.Property(p =>
p.InvoiceDate).HasColumnName("InvoiceDate").HasColumnType("datetimeoffset").IsRequir
ed();
                b.Property(p =>
p.DueDate).HasColumnName("DueDate").HasColumnType("datetimeoffset").IsRequired();
                b.Property(p =>
p.Status).HasColumnName("Status").HasMaxLength(16).HasConversion(
                    v => v.ToString(),
                    v => (InvoiceStatus)Enum.Parse(typeof(InvoiceStatus), v)
                );
                // b.HasMany(x => x.InvoiceItems).WithOne(x =>
x.Invoice).HasForeignKey(x => x.InvoiceId);
                b.OwnsMany(p => p.InvoiceItems, a =>
                {
                    a.WithOwner(x => x.Invoice).HasForeignKey(x => x.InvoiceId);
                    a.ToTable("InvoiceItems");
                });
            });
        }
    }
```

****

When you use the OwnsOne()/WithOwner() methods, you do not need to specify the foreign key property because the owned entity type will be stored in the same table as the owner by default. You can use the ToTable method to specify the table name of the owned entity type.

****

What is the difference between normal one-to-one or one-to-many and owned entity types? There are some differences:

• You cannot create a DbSet property for an owned entity type. You can only use the DbSet property for the owner. That means you do not have any way to access the owned entity type directly. You must access the owned entity type through the owner.

• When you query the owner, the owned entity type will be included automatically. You do not need to use the Include() method to include the owned entity type explicitly.

****

By default, a new DbContext instance is created for each request, which is generally not a problem since it is a lightweight object that does not consume many resources. However, in a high-throughput application, the cost of setting up various internal services and objects for each DbContext instance can add up. To address this, EF Core provides a feature called DbContext pooling, which allows the DbContext instance to be reused across multiple requests.

To enable DbContext pooling, you can replace the AddDbContext() method with the AddDbContextPool() method. This resets the state of the DbContext instance when it is disposed of, stores it in a pool, and reuses it when a new request comes in. By reducing the cost of setting up the DbContext instance, DbContext pooling can significantly improve the performance of your application for high-throughput scenarios.

****

The AddDbContextPool() method takes a poolSize parameter, which specifies the maximum number of DbContext instances that can be stored in the pool. The default value is 1024, which is usually sufficient for most applications. If the pool is full, EF Core will start creating new DbContext instances as needed.

****

Important note:

For most applications, DbContext pooling is not necessary. You should enable DbContext pooling only if you have a high-throughput application. Therefore, before enabling DbContext pooling, it is important to test your application's performance with and without it to see whether there's any noticeable improvement.

****

When we call the context.Invoices.FindAsync(id) method for the first time, EF Core will query the database and return the Invoice entity. The second time, EF Core will return the Invoice entity from the context because the Invoice entity is already in the context.

****

But in rare cases, Find() and FindAsync() may return outdated data if the entity is updated in the database after it is loaded into the context.

****

For example, if you use the bulk-update ExecuteUpdateAsync() method, the update will not be tracked by DbContext. Then, if we use Find() or FindAsync() to get the entity from DbContext, you will get the outdated data. In this case, you should use Single() or SingleOrDefault() to get the entity from the database again. In most cases, you can use the Find() or FindAsync() methods to get an entity by its primary key when you are sure the entity is always tracked by the DbContext.

****

An entity has one of the following `EntityState` values: `Detached`, `Added`, `Unchanged`, `Modified`, or `Deleted`. We introduced the `EntityState` enum in *Chapter 5*. The following is how the states change:

- All the entities that are returned by the query (such as `Find()`, `Single()`, `First()`, `ToList()`, and their `async` overloads) are in the `Unchanged` state
- If you update the properties of the entity, EF Core will change the state to `Modified`
- If you call the `Remove()` method on the entity, EF Core will change the state to `Deleted`
- If you call the `Add()` method on the entity, EF Core will change the state to `Added`

- If you call the `Attach()` method on the untracked entity, EF Core will track the entity and set the state to `Unchanged`
- If you call the `Detach()` method on the tracked entity, EF Core will not track the entity and will change the state to `Detached`

****

By default, tracking is enabled in EF Core. However, there may be scenarios where you do not want EF Core to track changes to entities. For instance, in read-only queries within `Get` actions, where the `DbContext` only exists for the duration of the request, tracking is not necessary. Disabling tracking can enhance performance and save memory. If you don't intend to modify entities, you should disable tracking by calling the `AsNoTracking()` method on the query. Here's an example:

```
// To get the invoice without tracking
var invoice = await context.Invoices.AsNoTracking().
FirstOrDefaultAsync(x => x.Id == id);
// To return a list of invoices without tracking
var invoices = await context.Invoices.AsNoTracking().ToListAsync();
```

****

If you have lots of read-only queries and you feel it is tedious to call the `AsNoTracking()` method every time, you can disable tracking globally when you configure the `DbContext`. The following code shows how to do this:

```
protected override void OnConfiguring(DbContextOptionsBuilder
optionsBuilder)
{
    base.OnConfiguring(optionsBuilder);
    optionsBuilder.UseQueryTrackingBehavior(QueryTrackingBehavior.
NoTracking);
}
```

****

For any other queries that you want to track, you can call the `AsTracking()` method on the query, as shown in the following code:

```
// To get the invoice with tracking
var invoice = await context.Invoices.AsTracking().
FirstOrDefaultAsync(x => x.Id == id);
// To return a list of invoices with tracking
var invoices = await context.Invoices.AsTracking().ToListAsync();
```

In the preceding code, we explicitly call the `AsTracking()` method to enable tracking for the query, so that we can update the entity and save the changes to the database.

****

If an entity is a keyless entity, EF Core will never track it. Keyless entity types do not have keys defined on them. They are configured by a [Keyless] data annotation or a Fluent API HasNoKey() method. The keyless entity is often used for read-only queries or views.

****

The first difference between `IQueryable` and `IEnumerable` is that `IQueryable` is in the `System.Linq` namespace, while `IEnumerable` is in the `System.Collections` namespace. The `IQueryable` interface inherits from the `IEnumerable` interface, so `IQueryable` can do everything that `IEnumerable` does. But why do we need the `IQueryable` interface?

One of the key differences between `IQueryable` and `IEnumerable` is that `IQueryable` is used to query data from a specific data source, such as a database. `IEnumerable` is used to iterate through a collection in memory. When we use `IQueryable`, the query will be translated into a specific query language, such as SQL, and executed against the data source to get the results when we call the `ToList()` (or `ToAway()`) method or iterate the items in the collection.

****

```
// Use IEnumerable
logger.LogInformation($"Creating the IEnumerable...");
var list2 = context.Invoices.Where(x => status == null || x.Status ==
status).AsEnumerable();
logger.LogInformation($"IEnumerable created");
logger.LogInformation($"Query the result using IEnumerable...");
var query2 = list2.OrderByDescending(x => x.InvoiceDate)
    .Skip((page - 1) * pageSize)
    .Take(pageSize);
logger.LogInformation($"Execute the query using IEnumerable");
var result2 = query2.ToList();
logger.LogInformation($"Result created using IEnumerable");
```

****

Now, we can see the difference between the two interfaces. The `IQueryable` interface is a deferred execution query, which means the query is not executed when we add more conditions to the query. The query will be executed against the database when we call the `ToList()` or `ToArray()` methods or iterate the items in the collection. So, in complex and heavy queries, we should always use the `IQueryable` interface to avoid fetching all the data from the database. Be careful when you call the `ToList()` or `ToArray()` methods because `ToList()` or `ToArray()` (and their `async` overloads) will execute the query immediately.

What LINQ methods can cause the query to be executed immediately?

There are a couple of operations that result in the query being executed immediately:

- Use the `for` or `foreach` loop to iterate the items in the collection
- Use the `ToList()`, `ToArray()`, `Single()`, `SingleOrDefault()`, `First()`, `FirstOrDefault()`, or `Count()` methods, or the `async` overloads of these methods

****

```
private static decimal CalculateTax(decimal amount)
{
    return amount * 0.15m;
}
```

```
var list = await context.Invoices
    .Where(x => x.ContactName.Contains(search) || x.InvoiceNumber.
Contains(search))
    .Select(x => new Invoice
    {
```



```
        Id = x.Id,
        InvoiceNumber = x.InvoiceNumber,
        ContactName = x.ContactName,
        Description = $"Tax: ${CalculateTax(x.Amount)}.
 {x.Description}",
        Amount = x.Amount,
        InvoiceDate = x.InvoiceDate,
        DueDate = x.DueDate,
        Status = x.Status
    })
    .ToListAsync();
```

****

Why must the CalculateTax() method be static?

EF Core caches the compiled query due to the expensive nature of compiling the query. If the `CalculateTax()` method is not static, EF Core will need to maintain a reference to a constant expression of the `InvoicesController` through the `CalculateTax()` instance method, which could potentially lead to memory leaks. To prevent this, EF Core throws an exception if the `CalculateTax()` method is not static. Making the method static will ensure that EF Core does not capture constant in the instance.

****

EF Core provides several methods to execute raw SQL queries:

- `FromSql()`
- `FromSqlRaw()`
- `SqlQuery()`
- `SqlQueryRaw()`
- `ExecuteSql()`
- `ExecuteSqlRaw()`

****

```
[HttpGet]
[Route("status")]
public async Task<ActionResult<IEnumerable<Invoice>>>
GetInvoices(string status)
{
    // Omitted for brevity
    var list = await context.Invoices
        .FromSql($"SELECT * FROM Invoices WHERE Status = {status}")
        .ToListAsync();
    return list;
}
```

****

The parameter is not inserted into the SQL query directly. Instead, EF Core uses the @p0 parameter placeholder and passes the parameter value to the SQL query. This is called a parameterized query. It is safe to use the parameterized query to avoid SQL injection attacks. So, we do not need to worry about the safety of the FromSql method.

****

Why FromSql() is safe to use

The `FromSql()` method expects a parameter as the `FormattableString` type. So, it is required to use the interpolated string syntax by using the `$` prefix. The syntax looks like regular C# string interpolation, but it is not the same thing. A `FormattableString` type can include interpolated parameter placeholders. The interpolated parameter values will be automatically converted to the `DbParameter` type. So, it is safe to use the `FromSql()` method to avoid SQL injection attacks.

****

Note that the SqlQuery() method is used on the Database property of the DbContext object. It is not available on the DbSet object.

SqlQuery() and SqlQueryRaw()

The `FromSql()` method is useful when we want to query entities from the database using a raw SQL query. For some cases, we want to execute the raw SQL query and return a scalar value or non-entity type. For example, we want to query the IDs of invoices that have a specific status. We can use the `SqlQuery()` method to execute the raw SQL query and return a list of IDs. Here is an example:

```
[HttpGet]
[Route("ids")]
public ActionResult<IEnumerable<Guid>> GetInvoicesIds(string status)
{
    var result = context.Database
        .SqlQuery<Guid>($"SELECT Id FROM Invoices WHERE Status =
{status}")
        .ToList();
    return result;
}
```

****

ExecuteSql() and ExecuteSqlRaw()

For some scenarios, where we do not need return values, we can use the `ExecuteSql` method to execute a raw SQL query. Normally, it is used to update or delete data or call a **stored procedure**. For example, when we need to delete all invoices that have a specific status, we can use the `ExecuteSql()` method to execute the raw SQL query. Here is an example:

```
[HttpDelete]
[Route("status")]
public async Task<ActionResult> DeleteInvoices(string status)
{
    var result = await context.Database
        .ExecuteSqlAsync($"DELETE FROM Invoices WHERE Status =
{status}");
    return Ok();
}
```


****